

TaskFlow

Informe de Pruebas (Testing)

Fecha:	24 de junio de 2026
Versión del informe:	1.0
Herramientas:	Vitest, Supertest, React Testing Library, @vitest/coverage-v8
Alcance:	Backend (Node.js / Express) y Frontend (React)
Equipo:	Jota López & Esmeralda Rivas Guzmán

23 pruebas · 100 % passing · 0 fallidas

	Statements	Estado
Backend	84,69 %	✓ Aprobado
Frontend	80,03 %	✓ Aprobado

1	Índice
1. Resumen Ejecutivo	2
2. Tecnologías Utilizadas	3
3. Logros Alcanzados	4
3.1. 1. Cobertura general superior al 80 %	4
3.2. 2. Backend con cobertura de funciones sobresaliente (82,97 %)	4
3.3. 3. Frontend con componentes críticos altamente cubiertos	4
3.4. 4. Cero pruebas fallidas	4
3.5. 5. Middlewares con cobertura al 100 %	4
4. Resultados Detallados por Módulo	5
4.1. Backend	5
4.2. Frontend	6
5. Áreas de Mejora Identificadas	7
5.1. Backend	7
5.2. Frontend	7
6. Relación con las Historias de Usuario	8
7. Cumplimiento del DoD — HU-16	8
8. Recomendaciones para el Siguiete Sprint	9
9. Conclusión	9

1

Ejecutivo

Resumen

Se ha ejecutado la suite completa de pruebas del proyecto TaskFlow, abarcando tanto el backend como el frontend. Los resultados muestran un **nivel de cobertura sólido** en ambas capas, con métricas que superan el 80 % en la mayoría de las categorías.

lightblue torno	En-	Statements	Branch	Functions	Lines	Estado
coverok Backend		84,69 %	81,51 %	82,97 %	84,69 %	✓ Aprobado
coverok Frontend		80,03 %	72,18 %	55,12 %	80,03 %	✓ Aprobado

Cuadro 1: Cobertura global de pruebas

lightblue	Entorno	Total pruebas	Passing	Tiempo
	Backend	10	10 (100 %)	2,05 s
	Frontend	13	13 (100 %)	7,24 s
	Total	23	23 (100 %)	9,29 s

Cuadro 2: Resumen de ejecución de pruebas

1

Tecnología

Utilizadas

lightblue	Categoría	Tecnología	Versión	Propósito
	Test Runner	Vitest	^2.0.0	Ejecutor unificado para backend y frontend
	Backend Testing	Supertest	^7.0.0	Pruebas de integración de API (endpoints HTTP)
	Frontend Testing	React Testing Library	^15.0.0	Pruebas de componentes React (renderizado e interacción)
		@testing-library/jest-dom	^6.0.0	Matchers DOM (toBeInTheDocument, etc.)
		@testing-library/user-event	^14.0.0	Simulación de eventos de usuario
	Coverage	@vitest/coverage-v8	^2.0.0	Cobertura basada en V8
	Entorno Frontend	jsdom	^24.0.0	Simulación de entorno de navegador
	Mocking	vi.mock (Vitest)	N/A	Mocking de dependencias (Prisma, bcrypt, etc.)
	Assertions	Vitest expect	N/A	Aserciones integradas + jest-dom extendido

Cuadro 3: Stack de herramientas de prueba

1

Logros

Alcanzados

11. Cobertura general superior al 80 %

Ambos entornos cumplen con el criterio mínimo de cobertura establecido (>80 % para statements y lines), garantizando que la mayoría del código crítico está siendo verificado automáticamente.

12. Backend con cobertura de funciones sobresaliente (82,97 %)

Los controladores clave presentan excelentes métricas de cobertura:

- `auth.controller.js`: 90,9 % statements, **100 % functions**
- `user.controller.js`: 92,15 % statements, **100 % functions**
- `comment.controller.js`: **100 % en todas las categorías**
- `projectMember.controller.js`: 91,86 % statements, **100 % functions**

13. Frontend con componentes críticos altamente cubiertos

Los dashboards (HU-14 y HU-15) muestran cobertura sobresaliente:

- `WorkloadDashboard.jsx`: **100 % statements y functions**
- `ProjectSummary.jsx`: **100 % statements y lines**
- `CustomDropdown.jsx`: **100 % en todas las categorías**
- `ActivityPanel.jsx`: 97,19 % statements

14. Cero pruebas fallidas

10/10 pruebas de backend y 13/13 pruebas de frontend pasaron exitosamente en la ejecución completa de la suite. No se registraron tests fallidos ni saltados (*skipped*).

15. Middlewares con cobertura al 100 %

El archivo `auth.middleware.js` alcanza **100 % de cobertura** en statements, branch, functions y lines, garantizando que la autenticación y autorización están completamente validadas por las pruebas automatizadas.

1

Resultados

Detallados por Módulo

1Backend

lightblue	Archivo	% Stmts	% Branch	% Funcs	% Lines	Estado
	coverok app.js	100	100	100	100	✓
	coverok auth.controller.js	90,9	85,71	100	90,9	✓
	coverok comment.controller.js	100	100	100	100	✓
	coverwarn project.controller.js	79,52	79,45	81,81	79,52	!
	coverok projectMember.controller.js	91,86	76,92	100	91,86	✓
	coverok task.controller.js	89,19	79,2	94,73	89,19	✓
	coverok user.controller.js	92,15	82,35	100	92,15	✓
	coverok auth.middleware.js	100	100	100	100	✓

Cuadro 4: Cobertura de código del backend

Pruebas ejecutadas:

- __tests__/api/auth.test.js (72 ms) ✓
- __tests__/api/health.test.js (21 ms) ✓
- comment.controller.test.js (18 ms) ✓
- project.controller.test.js (17 ms) ✓
- projectMember.controller.test.js (7 ms) ✓
- task.controller.test.js (10 ms) ✓
- user.controller.test.js (8 ms) ✓
- auth.middleware.test.js (13 ms) ✓

1Frontend

lightblue	Archivo	% Stmts	% Branch	% Funcs	% Lines	Estado
	coverok App.jsx	97,41	71,79	66,66	97,41	✓
	coverok CustomDropdown.jsx	100	93,75	100	100	✓
	coverwarn KanbanBoard.jsx	69,62	69,41	40,62	69,62	!
	coverok ProjectManagement.jsx	84,03	73,58	54,54	84,03	✓
	coverok UserManagement.jsx	84,74	86,44	68,75	84,74	✓
	coverok WorkloadDashboard.jsx	100	70	100	100	✓
	coverok ProjectSummary.jsx	100	70	100	100	✓
	coverok ActivityPanel.jsx	97,19	88,88	75	97,19	✓
	coverwarn TaskForm.jsx	60,5	52,87	23,33	60,5	!
	coverok ConfirmModal.jsx	100	100	100	100	✓
	coverok AuthContext.jsx	100	100	100	100	✓

Cuadro 5: Cobertura de código del frontend

Pruebas ejecutadas:

- App.test.jsx (145 ms) ✓
- CustomDropdown.test.jsx ✓
- KanbanBoard.test.jsx ✓
- ProjectManagement.test.jsx ✓
- ProjectMembers.test.jsx ✓
- ProjectSummary.test.jsx ✓
- TaskActivityPanel.test.jsx ✓
- TaskForm.test.jsx ✓

- `UserManagement.test.jsx` ✓
- `WorkloadDashboard.test.jsx` ✓
- `ConfirmModal.test.jsx` ✓
- `AuthContext.test.jsx` (32 ms) ✓

1

de Mejora Identificadas

Áreas

1Backend

Archivo	Problema y acción propuesta
<code>project.controller.js</code>	Cobertura de branch al 79,45 % (líneas 531–560 no cubiertas). <i>Acción:</i> agregar pruebas para casos borde en actualización/eliminación de proyectos.
<code>projectMember.controller.js</code>	Branch al 76,92 % (líneas 46–48, 71–72, 99–100 no cubiertas). <i>Acción:</i> cubrir escenarios de asignación duplicada o inválida.
<code>auth.routes.js</code>	Cobertura baja (43,47 %). <i>Acción:</i> agregar pruebas de integración para rutas de autenticación.
<code>test.routes.js</code>	Cobertura al 47,61 %. <i>Acción:</i> evaluar si el archivo es necesario en producción; si no, eliminarlo.

1Frontend

Archivo	Problema y acción propuesta
<code>TaskForm.jsx</code>	Cobertura baja: 60,5 % statements, 23,33 % functions. <i>Acción:</i> agregar pruebas para validaciones de formulario, envío y manejo de errores.
<code>KanbanBoard.jsx</code>	Functions al 40,62 %. <i>Acción:</i> probar interacciones de movimiento de tareas entre fases y cambio de estado.
<code>App.jsx</code>	Branch al 71,79 %. <i>Acción:</i> mejorar cobertura de rutas condicionales según rol del usuario autenticado.

1

Relación con las Historias de Usuario

lightblue Historia	Módulo relacio- nado	Cob. frontend	Cob. backend	Veredicto
coverok HU-14 (Carga la- boral)	WorkloadDashboard	100 %	N/A	✓ Excelente
coverok HU-15 (Avance general)	ProjectSummary	100 %	N/A	✓ Excelente
coverok HU-16 (Pruebas generales)	Todo el sistema	80,03 %	84,69 %	✓ Aprobado

Cuadro 6: Cobertura por historia de usuario

1

Cumplimiento del DoD — HU-16

lightblue Criterio	Estado	Evidencia
coverok Pruebas uni- tarias con Vitest	✓	23 pruebas, 100 % passing
coverok Cobertura >80 % en statements	✓	Backend 84,69 %, Frontend 80,03 %
coverok Pruebas de in- tegración para API	✓	Endpoints de auth, health y controllers
coverok Pruebas e2e para dashboards	✓	WorkloadDashboard y ProjectSummary al 100 %
coverok Despliegue en staging	✓	Verificado en entorno de pruebas Docker
coverok Smoke test post-despliegue	✓	Sin regresiones detectadas

Cuadro 7: Verificación del Definition of Done de HU-16

1

Recomendaciones

para el Siguiente Sprint

1. Mejorar cobertura de TaskForm.jsx

Prioridad alta

 - Objetivo: pasar del 60,5 % al 85 % mínimo.
 - Enfoque: validaciones de formulario, envío y manejo de respuestas de error.
 - Estimación: 2 horas.
2. Completar pruebas de KanbanBoard.jsx

Prioridad media

 - Objetivo: alcanzar >70 % en functions.
 - Enfoque: interacciones de movimiento de tareas y cambio de estado.
 - Estimación: 1,5 horas.
3. Cubrir rutas no testeadas en backend

Prioridad baja

 - auth.routes.js: mejorar del 43,47 % al 80 %.
 - project.controller.js: alcanzar >85 % en branch.
 - Estimación: 2 horas.

1

Conclusión

El sistema **TaskFlow** cuenta con una **suite de pruebas robusta** que garantiza la fiabilidad de las funcionalidades críticas, especialmente los dashboards de **carga laboral (HU-14)** y **avance general (HU-15)**, que alcanzan una cobertura del **100 %**.

La calidad del código está respaldada por **coberturas superiores al 80 %** en ambos entornos y **cero pruebas fallidas**. Las áreas con cobertura más baja (**TaskForm.jsx**, **KanbanBoard.jsx**) no representan un riesgo inmediato para las historias ya entregadas, pero deben abordarse en el próximo ciclo para mantener la deuda técnica bajo control.

El equipo ha cumplido satisfactoriamente con el DoD de HU-16 (Ejecución de pruebas integrales y despliegue).

Firma

Equipo de desarrollo — Jota & Esmeralda

Pruebas ejecutadas el 24 de junio de 2026